Report IUM-TWEB

Bissal Youssef, Lombardozzi Christian

Introduction: 1
Code Structure: 1
Express server: 2
Spring Boot server: 3
Frontend (express server) 4
Division of work: 5
Conclusion: 5

Introduction:

In this report we will explain all the main features of our site "QuieroMovies" and how they work. The website is like a bookshelf for movies, where the users can find their favourite movies, they can also search with several filters, for example: by name, by year, by genre, by minimum rating…
These searches reach the database,or to be more precise the databases (one is Postgres, one is MongoDb) that will respond to the http request.

Code Structure:

We organised the project in this way:
-MainDirectory
-TWEB

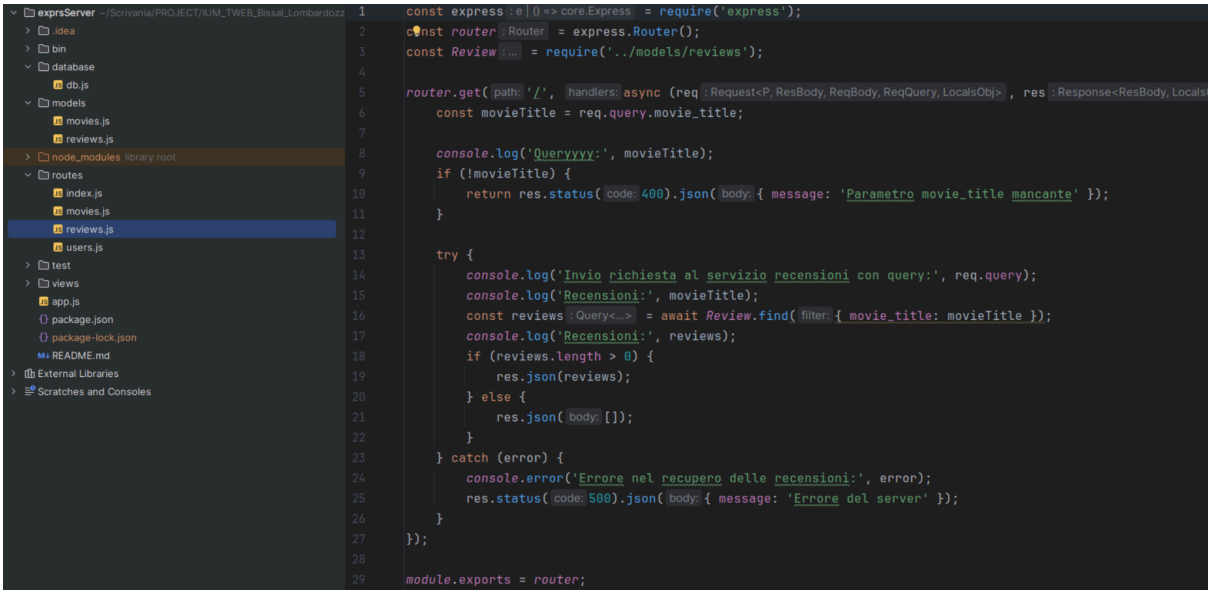
- exprsServer →This is a backend server, and it handles the request for the Mongo database
- springbootServer → This is a backend server and it handles the request for postgres database
- WebApp_TWEB → This is the frontend server and it send http request, using axios, to the backend, then expose the results to the users, filling the handlebars file with the correct data

Backend:

Here the discussion of the two backend Servers:

Express Server:

This server communicate with a Mongo database, at address: mongodb://localhost:27017/iumtweb.
In particular this server extracts the reviews about the movie selected in the frontend. We use MongoDB(non_relational database) because it is perfect for data that updates frequently, like reviews.

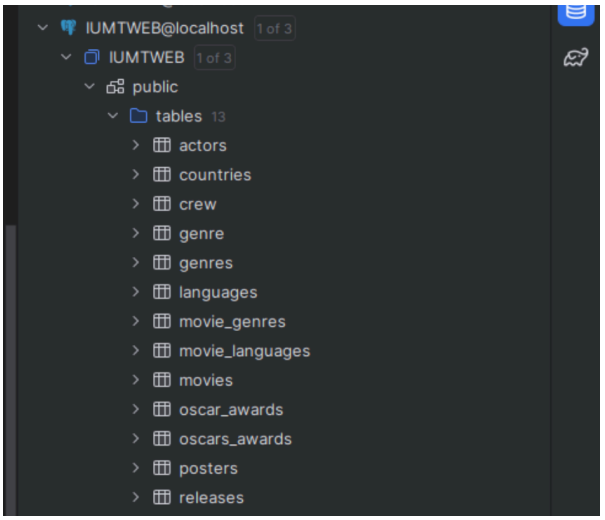


Explaining directories:

- models: Inside the models directory we define the schema of the collections in the database
- db: Here is a single file that starts the connection with the database, and handles possible errors.
- test: Inside there are a few tests for connection and for querying the database, the tests are self contained.
- routes: Here there is the main part of the server, where we receive the request from the frontend, and manage to respond.

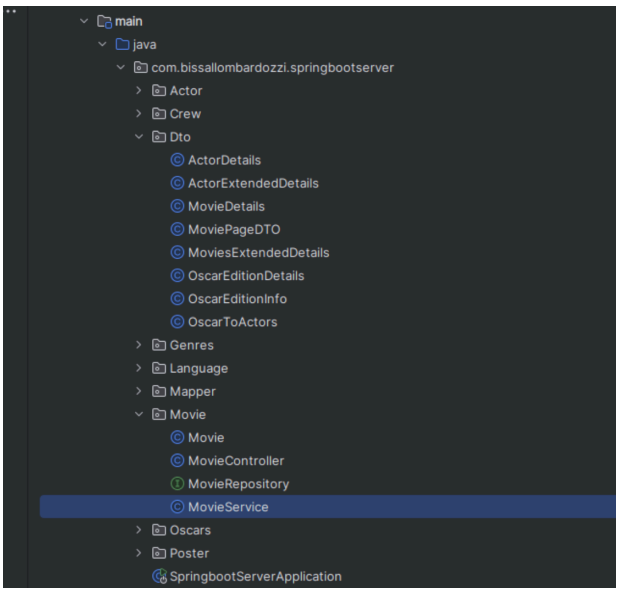
Spring Boot server:

This server instead communicates with the postgres database. This server communicates with the majority of the entities we have, because this kind of data tends to remain stable.



Explaining directories:

We have a directories for all the entities we use, and inside of it there is the logic to handle the request from the frontend, query the database, and build the DTO (data transfer object) to return, the following picture shows the structure.



To avoid exposing the database to the frontend, and also to build a more complex object to return as "MovieExtendedDetails", that contains information from several tables in db, we decided to create all these DTOs and return this custom object to the client. The code results are very organised, each file has his specific task.

Features:

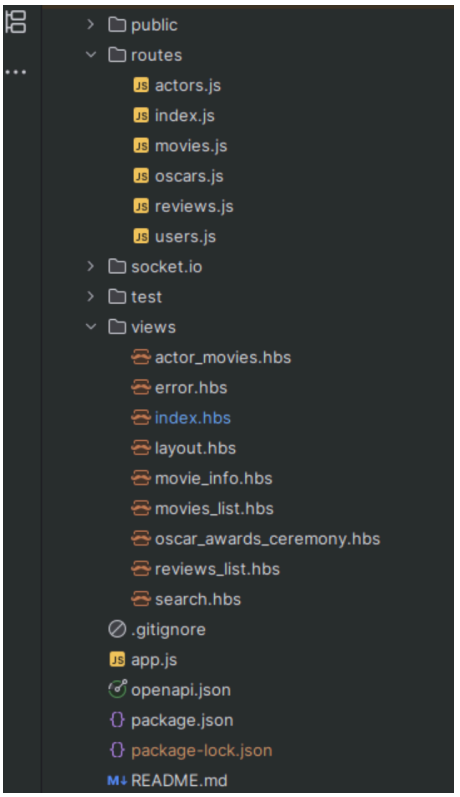
- To maximize the speed of the response we built two different DTOs for movies: "MovieDetails" and "MovieExtendedDetails". The first one is used when from the frontend we search for a list of movies, and it contains only the main info. The second one is used when the client requests

a specific movie page, and it contains a lot more information such as the genre, cast, the language…

- We use JPA annotations for join tables with hibernate and even for search on tables, useful to abstract from the SQL language.
- This server also handles the request of a "actorPage" and it will returns:
  -The list of movies where the actor/actress appears.
  -The list of awards won if present.

Frontend (express server)

At the end this server exposes the website of PORT: 3000 and using handlebars it can produce a modular web page that has always the same format and style, but the data depends on the users requests.
The website collects the input from users and it calls the correct route to get the information requested.
The route uses axios to communicate with the backend servers, wait for response then render the response in an appropriate format to the web page, Errors are handled properly, with message and code status.



Features:

- From a movie page it is possible to access an actor page by clicking on actor name.
- In all the website there is always a sticky button which allows the user go back to the Home page
- Documentation in done in Swagger

- The frontend is very fast and delegates heavy work to the backend.

Division of work:

The workload is splitted 50/50, we worked on each file of this project, often we worked together using “code with me” or google meet.

Conclusion:

This part of the assignment was a bit harder due the complications of handling a servers constellation, and even all the different technologies we used.
So for this reason the code must be very organised, to better understand what we were doing. We also try to manage efficiency problems, avoiding the bottleneck issue.